```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("andrewmvd/ocular-disease-recognition-odir5k")

print("Path to dataset files:", path)
```

```
Using Colab cache for faster access to the 'ocular-disease-recognition-odir5k' dataset.
Path to dataset files: /kaggle/input/ocular-disease-recognition-odir5k
```

```
import os

# check the root folder of the dataset
print(os.listdir("/kaggle/input/ocular-disease-recognition-odir5k/preprocessed_images")[:5])
```

```
['3419_left.jpg', '4176_right.jpg', '3370_left.jpg', '1255_right.jpg', '660_left.jpg']
```

```
# check images in part 1
print(os.listdir("/kaggle/input/ocular-disease-recognition-odir5k/ODIR-5K/ODIR-5K/Training Images")[:5])

# check images in part 2
print(os.listdir("/kaggle/input/ocular-disease-recognition-odir5k/ODIR-5K/ODIR-5K/Testing Images")[:5])
```

```
['3419_left.jpg', '4176_right.jpg', '679_left.jpg', '3370_left.jpg', '1255_right.jpg']
['3552_right.jpg', '4711_left.jpg', '3534_right.jpg', '1911_right.jpg', '4691_left.jpg']
```

PREPROCESSING

```
import torch

print("GPU available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
```

```
GPU available: True
GPU name: Tesla T4
```

```
import pandas as pd

df = pd.read_csv("/kaggle/input/ocular-disease-recognition-odir5k/full_df.csv")
df.head()
```

	ID	Patient Age	Patient Sex	Left-Fundus	Right-Fundus	Left-Diagnostic Keywords	Right-Diagnostic Keywords	N	D	G	C	A	H	M	O	filepath	labels	target	filen
0	0	69	Female	0_left.jpg	0_right.jpg	cataract	normal fundus	0	0	0	1	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...	['N']	[1, 0, 0, 0, 0, 0, 0]	0_right
1	1	57	Male	1_left.jpg	1_right.jpg	normal fundus	normal fundus	1	0	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...	['N']	[1, 0, 0, 0, 0, 0, 0]	1_right
2	2	42	Male	2_left.jpg	2_right.jpg	laser spot, moderate non proliferative retinopathy	moderate non proliferative retinopathy	0	1	0	0	0	0	0	1	../input/ocular-disease-recognition-odir5k/ODI...	['D']	[0, 1, 0, 0, 0, 0, 0]	2_right
3	4	53	Male	4_left.jpg	4_right.jpg	macular epiretinal membrane	mild nonproliferative retinopathy	0	1	0	0	0	0	0	1	../input/ocular-disease-recognition-odir5k/ODI...	['D']	[0, 1, 0, 0, 0, 0, 0]	4_right
4	5	50	Female	5_left.jpg	5_right.jpg	moderate non proliferative	moderate non proliferative retinopathy	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-	['D']	[0, 1, 0, 0, 0, 0, 0]	5_right

```
# Columns representing diseases
disease_cols = ['N','D','G','C','A','H','M','O']

# Count number of patients per disease
disease_counts = df[disease_cols].sum().sort_values()

print("Number of patients per disease (ascending order):")
print(disease_counts)
```

```
Number of patients per disease (ascending order):
H     203
M     306
A     319
G     397
C     402
O    1588
N    2101
D    2123
dtype: int64
```

```
df.isnull().sum()
```

	0
ID	0
Patient Age	0
Patient Sex	0
Left-Fundus	0
Right-Fundus	0
Left-Diagnostic Keywords	0
Right-Diagnostic Keywords	0
N	0
D	0
G	0
C	0
A	0
H	0
M	0
O	0
filepath	0
labels	0
target	0
filename	0

dtype: int64

```
df.drop(columns=[   'ID',   'Patient Age',  'Patient Sex',  'Left-Fundus',  'Right-Fundus', 'Left-Diagnostic Keywords', 'Right-
```

	N	D	G	C	A	H	M	O	filepath
0	0	0	0	1	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
1	1	0	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
2	0	1	0	0	0	0	0	1	../input/ocular-disease-recognition-odir5k/ODI...
3	0	1	0	0	0	0	0	1	../input/ocular-disease-recognition-odir5k/ODI...
4	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
...	...	...	...	...	...	...	...	...	...
6387	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
6388	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
6389	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
6390	0	1	0	0	0	0	0	0	../input/ocular-disease-recognition-odir5k/ODI...
6391	0	0	0	0	1	1	0	0	../input/ocular-disease-recognition-odir5k/ODI...

6392 rows × 9 columns

```
IMAGE_DIR = "/kaggle/input/ocular-disease-recognition-odir5k/preprocessed_images"

existing_images = os.listdir(IMAGE_DIR)
existing_images[:10], len(existing_images)
```

```
(['3419_left.jpg',
  '4176_right.jpg',
  '3370_left.jpg',
  '1255_right.jpg',
  '660_left.jpg',
  '484_right.jpg',
  '4221_right.jpg',
  '2396_left.jpg',
  '543_left.jpg',
  '3017_left.jpg'],
 6392)
```

```
IMAGE_DIR = "/kaggle/input/ocular-disease-recognition-odir5k/preprocessed_images"

# IMPORTANT: extract only the filename
df['filepath'] = df['filename'].apply(
    lambda x: os.path.join(IMAGE_DIR, os.path.basename(x))
)
```

```
missing = [p for p in df['filepath'].values if not os.path.exists(p)]
len(missing)
```

```
0
```

```
from sklearn.model_selection import train_test_split

X = df['filepath'].values
y = df[['N','D','G','C','A','H','M','O']].values

X_train, X_val, y_train, y_val = train_test_split(
```

```
    X,
    y,
    test_size=0.30,   # 30% validation
    random_state=42,
    shuffle=True
)
```

```
len(X_train)
```

```
4474
```

```
len(X_val)
```

```
1918
```

AUGMENTATION

```
import torchvision.transforms as transforms

# 🔹 Training transforms (Resize + Normalize + Augmentation)
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),                # Step 1: Resize images
    transforms.RandomHorizontalFlip(p=0.5),       # Step 2: Horizontal flip
    transforms.RandomRotation(15),                # Step 3: Small rotations
    transforms.ColorJitter(brightness=0.1,
                           contrast=0.1),         # Step 4: Light intensity changes
    transforms.ToTensor(),                        # Step 5: Convert to tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406],# Step 6: Normalize (ImageNet)
                         std=[0.229, 0.224, 0.225])
])

# 🔹 Validation / Test transforms (NO augmentation)
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),                # Step 1: Resize images
    transforms.ToTensor(),                        # Step 2: Convert to tensor
    transforms.Normalize(mean=[0.485, 0.456, 0.406],# Step 3: Normalize
                         std=[0.229, 0.224, 0.225])
])
```

DATASET

```
import torch
from torch.utils.data import Dataset
from PIL import Image
```

```
class FundusDataset(Dataset):
    def __init__(self, image_paths, labels, transform=None): #constructor that contains image loc, labels and augmnetaion pipel
        self.image_paths = image_paths
        self.labels = labels
        self.transform = transform

    def __len__(self): # retuen dataset size as PyTorch needs this to know how many samples exist, decide when one epoch ends,

        return len(self.image_paths)

    def __getitem__(self, idx): # Defines how to fetch ONE data sample.
        # Load image
        image = Image.open(self.image_paths[idx]).convert("RGB")

        # Get labels
        label = torch.tensor(self.labels[idx], dtype=torch.float32) # float is required for BCEWithLogitsLoss / focal / asymmet

        # Apply transform
        if self.transform:
            image = self.transform(image)

        return image, label
```

DATALOADER

```
from torch.utils.data import DataLoader

# Create Dataset objects
train_dataset = FundusDataset(X_train, y_train, transform=train_transform)
test_dataset  = FundusDataset(X_val, y_val, transform=test_transform)

# Create DataLoaders
train_loader = DataLoader(
    train_dataset,
    batch_size=16, #with batch size 16 model will learn the avg trend and will not oversmooth results
    shuffle=True,
    num_workers=2 #num_workers controls how many CPU processes load and preprocess data in parallel, preventing the GPU from be
)

test_loader = DataLoader(
    test_dataset,
    batch_size=16,
    shuffle=False,
    num_workers=2
)

print("Train batches:", len(train_loader))
print("Test batches:", len(test_loader))
```

```
Train batches: 280
Test batches: 120
```

MODEL ARCHITECTURE

CBAM

```
import torch
import torch.nn as nn
import torch.nn.functional as F
```

```
class ChannelAttention(nn.Module):
    def __init__(self, in_channels, reduction=16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)

        self.fc = nn.Sequential(
            nn.Linear(in_channels, in_channels // reduction, bias=False),
            nn.ReLU(),
            nn.Linear(in_channels // reduction, in_channels, bias=False)
        )
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        b, c, _, _ = x.size()
        avg_out = self.fc(self.avg_pool(x).view(b, c))
        max_out = self.fc(self.max_pool(x).view(b, c))
        out = avg_out + max_out
        return x * self.sigmoid(out).view(b, c, 1, 1)
```

```
class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        self.conv = nn.Conv2d(2, 1, kernel_size, padding=kernel_size//2, bias=False)
        self.sigmoid = nn.Sigmoid()

    def forward(self, x):
        avg_out = torch.mean(x, dim=1, keepdim=True)
        max_out, _ = torch.max(x, dim=1, keepdim=True)
        x_cat = torch.cat([avg_out, max_out], dim=1)
        return x * self.sigmoid(self.conv(x_cat))
```

```
class CBAM(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.ca = ChannelAttention(channels)
        self.sa = SpatialAttention()

    def forward(self, x):
        x = self.ca(x)
        x = self.sa(x)
        return x
```

```
from torchvision import models
```

```
class EfficientNet_CBAM(nn.Module):
    def __init__(self, num_classes=8):
        super().__init__()

        # Load pretrained EfficientNet-B0
        self.backbone = models.efficientnet_b0(pretrained=True)

        # Extract feature dimension
        in_features = self.backbone.classifier[1].in_features

        # Remove default classifier
        self.backbone.classifier = nn.Identity()

        # CBAM block
        self.cbam = CBAM(in_features)

        # Classification head
        self.classifier = nn.Linear(in_features, num_classes)

    def forward(self, x):
        # Feature extraction
        x = self.backbone.features(x)     # (B, C, H, W)

        # Attention
        x = self.cbam(x)

        # Global Average Pooling
        x = F.adaptive_avg_pool2d(x, 1)
        x = x.view(x.size(0), -1)

        # Multi-label output (logits)
        x = self.classifier(x)

        return x
```

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

```
model = EfficientNet_CBAM(num_classes=8)
model = model.to(device)

print(model)
```

Show hidden output

```
#managing class imbalance
import torch
import numpy as np

label_cols = ['N','D','G','C','A','H','M','O']

# Convert labels to numpy
labels_np = df[label_cols].values

# Count positives per class
pos_counts = labels_np.sum(axis=0)

# Total samples
num_samples = labels_np.shape[0]

# Count negatives per class
neg_counts = num_samples - pos_counts
```

```
# pos_weight = negatives / positives
pos_weight = neg_counts / pos_counts

pos_weight = torch.tensor(pos_weight, dtype=torch.float32)

print("Positive counts :", pos_counts)
print("Negative counts :", neg_counts)
print("pos_weight      :", pos_weight)
```

```
Positive counts : [2101 2123  397  402  319  203  306 1588]
Negative counts : [4291 4269 5995 5990 6073 6189 6086 4804]
pos_weight      : tensor([ 2.0424,  2.0108, 15.1008, 14.9005, 19.0376, 30.4877, 19.8889,  3.0252])
```

```
#freezing layer in efficient net
for param in model.backbone.parameters():
    param.requires_grad = False
```

```
#updated loss function
criterion = nn.BCEWithLogitsLoss(pos_weight=pos_weight.to(device))

#optimizer (define Adam only for trainable parameters)
import torch.optim as optim

optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=3e-5,
    weight_decay=1e-4
)
```

```
def train_one_epoch(model, dataloader, criterion, optimizer, device):
    model.train()   # enable training mode
    total_loss = 0.0

    for images, labels in dataloader:
        images = images.to(device)
        labels = labels.to(device)

        # Forward
        outputs = model(images)
        loss = criterion(outputs, labels)

        # Backward
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    return avg_loss
```

```
def validate(model, dataloader, criterion, device):
    model.eval()
    total_loss = 0.0

    with torch.no_grad():
        for images, labels in dataloader:
            images = images.to(device)
            labels = labels.to(device)

            outputs = model(images)
            loss = criterion(outputs, labels)

            total_loss += loss.item()

    avg_loss = total_loss / len(dataloader)
    return avg_loss
```

```
num_epochs = 15
unfreeze=3
best_val_loss = float("inf")

for epoch in range(1, num_epochs + 1):
```

```
    print(f"\n🚀 Epoch {epoch}/{num_epochs}")

    if epoch==unfreeze:
        print("unfreezing")

    for param in model.backbone.parameters():
        param.requires_grad=True

    optimizer = optim.Adam(
    filter(lambda p: p.requires_grad, model.parameters()),
    lr=1e-5,
    weight_decay=1e-4 )


    train_loss = train_one_epoch(
        model,
        train_loader,
        criterion,
        optimizer,
        device
    )

    val_loss = validate(
        model,
        test_loader,
        criterion,
        device
    )

    print(f"Train Loss: {train_loss:.4f}")
    print(f"Val   Loss: {val_loss:.4f}")

    # Save best model
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        torch.save(model.state_dict(), "best_model.pth")
        print("✅ Best model saved")
```

```
🚀 Epoch 1/15
Train Loss: 1.1542
Val   Loss: 1.1657
✅ Best model saved

🚀 Epoch 2/15
Train Loss: 1.0892
Val   Loss: 1.0753
✅ Best model saved

🚀 Epoch 3/15
unfreezing
Train Loss: 1.0185
Val   Loss: 1.0099
✅ Best model saved

🚀 Epoch 4/15
Train Loss: 0.9695
Val   Loss: 0.9717
✅ Best model saved

🚀 Epoch 5/15
Train Loss: 0.9329
Val   Loss: 0.9436
✅ Best model saved

🚀 Epoch 6/15
Train Loss: 0.9000
Val   Loss: 0.9219
✅ Best model saved

🚀 Epoch 7/15
Train Loss: 0.8799
Val   Loss: 0.8957
✅ Best model saved

🚀 Epoch 8/15
Train Loss: 0.8507
Val   Loss: 0.8800
✅ Best model saved

🚀 Epoch 9/15
```

```
Train Loss: 0.8301
Val   Loss: 0.8609
✅ Best model saved

🚀 Epoch 10/15
Train Loss: 0.8142
Val   Loss: 0.8460
✅ Best model saved

🚀 Epoch 11/15
Train Loss: 0.8001
Val   Loss: 0.8378
✅ Best model saved

🚀 Epoch 12/15
```

```
torch.save({
    'epoch': epoch,
    'model_state': model.state_dict(),
    'optimizer_state': optimizer.state_dict(),
    'best_val_loss': best_val_loss
}, 'checkpoint.pth')
```

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

# Load the best model
best_model = EfficientNet_CBAM(num_classes=8)
best_model.load_state_dict(torch.load("best_model.pth"))
best_model = best_model.to(device)
best_model.eval()

all_preds = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        outputs = best_model(images)
        probabilities = torch.sigmoid(outputs)
        predicted = (probabilities > 0.5).int()

        all_preds.extend(predicted.cpu().numpy())
        all_labels.extend(labels.cpu().numpy())

all_preds = np.array(all_preds)
all_labels = np.array(all_labels)

# Calculate metrics for each class (multi-label classification)
metrics = {}
for i, col in enumerate(label_cols):
    accuracy = accuracy_score(all_labels[:, i], all_preds[:, i])
    precision = precision_score(all_labels[:, i], all_preds[:, i], zero_division=0)
    recall = recall_score(all_labels[:, i], all_preds[:, i], zero_division=0)
    f1 = f1_score(all_labels[:, i], all_preds[:, i], zero_division=0)

    metrics[col] = {
        'Accuracy': accuracy,
        'Precision': precision,
        'Recall': recall,
        'F1-Score': f1
    }

# Calculate overall metrics (micro-averaged for multi-label classification)
overall_accuracy = accuracy_score(all_labels, all_preds)
overall_precision = precision_score(all_labels, all_preds, average='micro', zero_division=0)
overall_recall = recall_score(all_labels, all_preds, average='micro', zero_division=0)
overall_f1 = f1_score(all_labels, all_preds, average='micro', zero_division=0)

print("\n--- Metrics Per Class ---")
for col, m in metrics.items():
    print(f"Class {col}:")
    print(f"  Accuracy: {m['Accuracy']:.4f}")
    print(f"  Precision: {m['Precision']:.4f}")
    print(f"  Recall: {m['Recall']:.4f}")
    print(f"  F1-Score: {m['F1-Score']:.4f}")

print("\n--- Overall Micro-Averaged Metrics ---")
print(f"Overall Accuracy: {overall_accuracy:.4f}")
```

```
print(f"Overall Precision: {overall_precision:.4f}")
print(f"Overall Recall: {overall_recall:.4f}")
print(f"Overall F1-Score: {overall_f1:.4f}")
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `No
  warnings.warn(msg)

--- Metrics Per Class ---
Class N:
  Accuracy: 0.6283
  Precision: 0.4411
  Recall: 0.7959
  F1-Score: 0.5676
Class D:
  Accuracy: 0.6710
  Precision: 0.5124
  Recall: 0.6937
  F1-Score: 0.5895
Class G:
  Accuracy: 0.8029
  Precision: 0.2343
  Recall: 0.8120
  F1-Score: 0.3636
Class C:
  Accuracy: 0.9301
  Precision: 0.4928
  Recall: 0.7863
  F1-Score: 0.6059
Class A:
  Accuracy: 0.8577
  Precision: 0.2190
  Recall: 0.7188
  F1-Score: 0.3358
Class H:
  Accuracy: 0.8217
  Precision: 0.1148
  Recall: 0.6119
  F1-Score: 0.1934
Class M:
  Accuracy: 0.9286
  Precision: 0.4112
  Recall: 0.8889
  F1-Score: 0.5623
Class O:
  Accuracy: 0.6283
  Precision: 0.3293
  Recall: 0.4447
  F1-Score: 0.3784

--- Overall Micro-Averaged Metrics ---
Overall Accuracy: 0.1517
Overall Precision: 0.3719
Overall Recall: 0.6860
Overall F1-Score: 0.4823
```